

LLD Practice Platform (MVP)

CipherSchools 2-Day Engineering Assignment

Author: Mainak Pal

A focused practice and evaluation platform for Low-Level Object-Oriented System Design (Parking Lot, Elevator, Vending Machine, Rate Limiter).

1. Quick Start Guide

Prerequisites

- **Node.js:** v18+ (tested on Node.js v24.16.0)
- **npm:** v9+ (tested on npm 11.13.0)

Running the Application

You can run both the backend API and the frontend client simultaneously in separate terminal windows:

Terminal 1: Backend Service (Port 4000)

```
cd backend
npm install
npm run dev
```

Backend runs at: <http://localhost:4000>

API Health check: <http://localhost:4000/health>

Terminal 2: Frontend Client (Port 3000)

```
cd frontend
npm install
npm run dev
```

Frontend runs at: <http://localhost:3000> (proxies `/api` requests to backend).

2. Running Automated Tests

A comprehensive unit and integration test suite verifies the domain models, state machines, evaluators, and edge cases:

```
cd backend
npm test
```

Test Coverage Highlights:

- **Attempt.test.ts** : Verifies the strict state machine transitions (**DRAFT** → **SUBMITTED** → **EVALUATING** → **COMPLETED** / **FAILED**), idempotency, and submission validation.
- **Evaluators.test.ts** : Verifies deterministic rule evaluation, heuristic/LLM cognitive analysis, and composite aggregation.
- **PracticeService.test.ts** : Tests attempt creation, submission flow, asynchronous runner, and progression delta calculation across attempts.
- **EdgeCases.test.ts** : Tests duplicate submission rejection (idempotency guard), evaluator failure recovery, and retry flow.

3. The Learner Journey (The Practice Loop)

The platform is designed around deliberate practice rather than passive reading:

[Choose Problem] → [Think - Model] → [Submit] → [Rubric Feedback] → [Review] → [Try Again (Progression)]

1. **Choose Problem:** Select from curated, archetypal LLD challenges:
 - **Parking Lot System** (Vehicle polymorphism, Spot allocation strategy, Dynamic pricing strategy, Concurrency).
 - **Elevator Control System** (State pattern, Internal vs. External calls, SCAN/LOOK dispatching).
 - **Vending Machine** (State transitions: Idle, HasMoney, Dispensing, SoldOut; Inventory rollback).
 - **Rate Limiter Library** (Token Bucket, Leaky Bucket, Sliding Window, Thread-safe atomic operations).
2. **Think & Model:** Define domain assumptions, render live Mermaid class diagrams, write class/interface skeletons, and document concurrency mechanisms.
3. **Submit:** Non-blocking async submission (**202 Accepted**).
4. **Rubric Feedback:** Receive actionable feedback structured in a fixed shape:

Criterion → Score → Evidence → Concern → Suggestion → Confidence

5. **Review & Try Again:** Re-attempt the problem and view an explicit **Progression Delta** comparing Attempt \$N\$ and Attempt \$N-1\$.

4. Architectural Highlights & Key Decisions

A. Monolithic Domain-Driven Design (No Microservice Bloat)

Per Section 5 of the brief, we avoid unnecessary Kubernetes, microservice, or Docker bloat. The system is built as a modular TypeScript monolith with decoupled domain boundaries:

- **domain/models**: Pure business entities (`Problem` , `Attempt` , `Submission` , `EvaluationResult` , `Rubric`).
- **domain/evaluators**: Strategy pattern implementation (`DeterministicRuleEvaluator` , `LlmEvaluator` , `CompositeEvaluator` , `HumanReviewEvaluatorStub`).
- **services**: `PracticeService` coordinating attempts, idempotency, and progression reports.
- **infrastructure**: Asynchronous `EvaluationJobRunner` providing non-blocking execution and failure resilience.

B. The Two Change Tests Answered

- **Change Test A (Diagram Support Later)**: Today the learner submits text/code. Later the platform supports visual diagram editing.
Impact: Zero changes to domain logic. The domain defines `ISubmissionContent` . We implement `StructuredTextSubmissionContent` . Adding a visual editor simply introduces another implementation of `ISubmissionContent` .
- **Change Test B (Pluggable Evaluators Later)**: Today feedback comes from one evaluator. Later rule-based evaluators or human reviewers are added.
Impact: Zero changes to practice flow. Handled via `CompositeEvaluator implements IEvaluator` . We include a live `HumanReviewEvaluatorStub` in the UI to demonstrate this extensibility.

5. Project Deliverables

Deliverable	File Location	Description
Research Note	RESEARCH_NOTE.md	1–2 page analysis of learner friction, existing tools, key gaps, and product direction.
Design Note	DESIGN_NOTE.md	Architectural specification, answers to the 5 design questions, and change test analysis.
AI Usage Report	AI_USAGE.md	4 critical AI-assisted decisions: what was suggested, accepted/rejected, and engineering rationale.
Backend Prototype & Tests	backend/	Domain model, evaluators, 17 passing tests, and Express REST API.
Frontend Prototype	frontend/	React + Tailwind application with live Mermaid preview, real-time polling, and progression diff.

6. Known Limitations & Production Roadmap

1. **Persistence:** The prototype uses in-memory repositories (`InMemoryProblemRepository` , `InMemoryAttemptRepository`). For production, replace with PostgreSQL or MongoDB using the same repository interfaces (`IProblemRepository` , `IAttemptRepository`).
 2. **Distributed Worker:** For 10,000+ concurrent learners, extract `EvaluationJobRunner` to a standalone Redis/BullMQ worker queue.
 3. **Live Remote LLM:** The system includes both offline heuristic rubric evaluation (zero setup required) and an optional remote LLM hook (`OPENAI_API_KEY` / `GEMINI_API_KEY`).
-

AI Usage Report (AI_USAGE.md)

Project: LLD Practice Platform MVP (CipherSchools 2-Day Engineering Assignment)

Author: Mainak Pal

Role: Candidate / Systems Architect

Executive Summary: Engineering Judgement vs. AI Suggestions

As encouraged by Section 6 of the assignment brief, AI assistance (Gemini / Antigravity) was engaged throughout this project as a **brainstorming partner and rapid prototyping accelerator**.

However, AI models suffer from known systemic biases when applied to software architecture:

1. **The Over-Engineering Bias:** Defaulting to distributed systems (Kafka, Docker, Microservices, Vector DBs) when a clean monolith is required.
2. **The "Sycophancy & Hallucination" Bias:** Handing out unearned 90+ scores and flattering praises instead of identifying subtle architectural anti-patterns.
3. **The "Pattern-itis" Anti-Pattern:** Cramming unnecessary design patterns (Visitor, Memento, Flyweight) where simple polymorphism or delegation suffices.
4. **The "Brittle Test Harness" Fallacy:** Assuming that low-level design can be verified with rigid unit tests checking exact class and method names.

To build a genuinely useful platform, **I actively audited, challenged, and rejected several key recommendations from the AI**. Below are **5 concrete architectural decisions** showcasing how my engineering judgement overruled the AI's suggestions.

Decision 1: Submission Format (Code Compilation vs. Structured Architectural Model)

- **What the AI Suggested:**

The AI strongly recommended creating a containerized sandbox (using Docker, isolated worker

containers, and an in-memory compiler) to compile student-written Java/TypeScript and run automated unit tests against their classes.

- **What Was Accepted:**

Accepted that learners should provide concrete class definitions, interfaces, and method signatures rather than pure high-level prose.

- **What I Rejected:**

I firmly rejected containerized compilation and rigid automated unit test harnesses.

- **Engineering Rationale & Why I Overruled the AI:**

1. *The Brittle Test Problem:* In real-world LLD, there is no single canonical class name or method signature. If a learner designs an elegant `SpotAllocationStrategy` with a method `findSpot(VehicleType)`, but our automated unit test expects `ParkingManager.allocate(SpotSize)`, the test suite fails—falsely penalizing a great design.
2. *Violation of Assignment Scope:* Spinning up Docker daemon wrappers, sandboxed runtime environments, and resource quotas is a heavy DevOps/HLD task that violates Section 5's mandate to focus strictly on domain design.
3. *The Solution Adopted:* I established a **Structured Architectural Model** (`ISubmissionContent`):
 - Explicit Domain Assumptions & Boundaries
 - Class/Interface Code Skeleton
 - Live Mermaid Class Diagram
 - Concurrency & Trade-offs This format captures the candidate's actual design decisions and abstractions without compiler brittleness.

Decision 2: Evaluation Design (Single Prompt "Score out of 100" vs. Fixed Rubric Shape with Evidence Anchoring)

- **What the AI Suggested:**

The AI suggested a simple, single-shot prompt sending the student's submission and problem description to an LLM with the instruction: *"Evaluate this low-level design, score it from 1 to 100, and give suggestions."*

- **What Was Accepted:**

Accepted using LLMs for semantic and cognitive reasoning (e.g. detecting Single Responsibility violations or assessing whether a class is becoming a "God Object").

- **What I Rejected:**

I completely rejected the single-prompt approach and the arbitrary 100-point score.

- **Engineering Rationale & Why I Overruled the AI:**

1. *Lack of Repeatability:* Unconstrained LLM prompts produce wildly fluctuating evaluations. Submitting the exact same design twice frequently yields different numbers (e.g., 76/100 followed by 89/100), destroying learner trust.
2. *Sycophancy:* Off-the-shelf LLMs tend to be overly polite, saying *"Great job! Your design is well-structured"* while overlooking critical design flaws like tight coupling or thread-safety race conditions.

3. *The Solution Adopted*: I designed a **Hybrid Composite Evaluation Pipeline**:

- **Deterministic First-Pass**: Regex/AST structural validation checks for minimum class decomposition, presence of interfaces, and edge-case handling.
- **Strict Rubric Shape**: Every criterion is strictly modeled as:

Criterion → Score → Direct Evidence → Concern → Suggestion → Confidence

- **Evidence Requirement**: The evaluator is forced to extract literal code tokens from the candidate's text as `evidence` (e.g., "*In `ParkingSpot.ts`, method `calculateFee()` directly accesses billing rules*"). Without concrete evidence, a concern cannot be raised.

Decision 3: System Architecture (Kafka + Microservices vs. Modular Domain Monolith)

- **What the AI Suggested:**

When asked how to handle slow or failing AI evaluations, the AI recommended an event-driven microservices architecture: an API Gateway, a submission microservice, an Apache Kafka or RabbitMQ event broker, separate Python evaluation workers, and a Redis state cache.

- **What Was Accepted:**

Accepted the principle of **asynchronous non-blocking evaluation** (`202 Accepted`) and the necessity of an explicit lifecycle state machine (`DRAFT` → `SUBMITTED` → `EVALUATING` → `COMPLETED` / `FAILED`).

- **What I Rejected:**

I rejected Kafka, RabbitMQ, and microservice decomposition entirely.

- **Engineering Rationale & Why I Overruled the AI:**

1. Section 5 & Section 10 of the brief explicitly warn: "*Do not turn the assignment into a distributed-systems project. A simple monolith is completely acceptable.*"
2. Introducing Kafka for an MVP is textbook resume-driven over-engineering. It adds serialization bugs, network latency, multi-process management, and deployment headaches without adding a single unit of value to the learner's feedback loop.
3. *The Solution Adopted*: Built a clean, thread-safe, in-process `EvaluationJobRunner` in TypeScript. Submissions are persisted immediately, evaluation executes asynchronously without blocking HTTP requests, duplicate retries are guarded by an idempotency check, and failures recover gracefully.

Decision 4: Design Pattern Appropriateness ("Pattern-itis" vs. Justified Pragmatism)

- **What the AI Suggested:**

When generating starter templates and evaluation rubrics, the AI attempted to inject 5–6 classical GoF design patterns into every problem: Abstract Factory, Visitor, Flyweight, Memento, and Observer for the *Parking Lot* and *Vending Machine*.

- **What Was Accepted:**

Accepted the value of classical design patterns when they solve an actual extensibility requirement.

- **What I Rejected:**

I removed the Visitor, Memento, and Flyweight patterns from the problem definitions and grading criteria.

- **Engineering Rationale & Why I Overruled the AI:**

1. *The "Pattern-itis" Anti-Pattern:* One of the biggest red flags in junior LLD interviews is applying patterns for the sake of showing off pattern vocabulary. A Parking Lot does not need the Visitor pattern to calculate hourly rates—doing so adds needless indirection and obfuscates the code.
2. *The Solution Adopted:* I focused the evaluation criteria strictly on **pragmatic extensibility**:
 - **Strategy Pattern** for spot allocation and dynamic pricing (where business algorithms genuinely vary).
 - **State Pattern** for the Elevator and Vending Machine (where valid actions depend strictly on internal finite states).
 - The rubric now explicitly rewards *simple, cohesive classes* and warns against speculative over-engineering.

Decision 5: The Learning Loop (Static Attempt History vs. Dynamic Progression Delta)

- **What the AI Suggested:**

The AI suggested a simple history view that just listed previous attempts in a basic table with timestamps and total scores.

- **What Was Accepted:**

Accepted that previous attempts must be retained and accessible to the learner.

- **What I Enhanced & Redesigned:**

I rejected a static history table and designed an active "Progression Delta Engine".

- **Engineering Rationale & Why I Overruled the AI:**

1. A list of scores does not answer the central question of deliberate practice: *"Did my design actually get better, and did I fix the specific flaw pointed out last time?"*
2. *The Solution Adopted:* Implemented domain-level progression diffing in [PracticeService.ts](#):
 - Compares Attempt \$N\$ against Attempt \$N-1\$.
 - Computes a criterion-by-criterion delta (e.g. *Class Responsibilities: \$+2.0\$, Concurrency: \$+3.0\$*).
 - Tracks whether previously identified concerns (e.g., tight coupling between [ParkingSpot](#) and [BillingManager](#)) were resolved in the latest submission.
 - Displays this delta visually in the UI to give learners immediate positive reinforcement.

Summary Matrix of AI Interactions

Decision Area	What AI Proposed	What I Decided	Engineering Justification
1. Submission Model	Docker sandbox + automated unit testing.	Structured Text + Live Mermaid Class Diagram.	Avoids brittle test naming; focuses on abstractions, relationships, and trade-offs.
2. Evaluation Engine	Single unconstrained prompt → 100-pt score.	Hybrid Composite Evaluator + Fixed Rubric Shape.	Eliminates AI sycophancy and score instability; requires token-level evidence.
3. Scalability / Jobs	Kafka + Microservices + Redis + Celery.	In-Process <code>EvaluationJobRunner</code> in Monolith.	Zero external bloat; satisfies Section 5 mandate while providing non-blocking async execution.
4. Design Patterns	Inject 6 GoF patterns per problem (Visitor, Memento, etc.).	Pragmatic patterns only (Strategy & State).	Penalizes over-engineering; rewards clean encapsulation and cohesion.
5. Attempt History	Static list of submission dates and scores.	Progression Delta Engine (Attempt $\\$N\\$ vs $\\$N-1\\$).	Deliberate practice requires tracking the exact resolution of architectural critique.